

Định vị lỗi mã nguồn bằng Hệ đa tác tử LLM

Kết hợp RAG và Code Property Graph (CPG)

Định vị file / method chứa lỗi từ bug report ngôn ngữ tự nhiên

Luận văn · Seminar trình bày 2026

Lộ trình trình bày

- I. Giới thiệu & động lực
- II. Cơ sở lý thuyết & công trình liên quan
- III. Kiến trúc hệ thống
- IV. Đóng góp phương pháp (E1 / E2 / E3)
- V. Thiết lập thí nghiệm
- VI. Kết quả & phân tích
- VII. Thảo luận & kết luận

Phần I

Giới thiệu & động lực

Bối cảnh: Bug localization là gì?

Định vị lỗi (Bug / Fault Localization) — nhận bug report + mã nguồn, trả về các vị trí (file/method) nghi ngờ chứa lỗi, theo thứ tự khả năng.

*"Debugging chiếm 30–50% tổng thời gian phát triển phần mềm, và phần lớn thời gian đó dành cho việc **tìm vị trí** lỗi chứ không phải sửa nó."*

Hai thách thức cốt lõi:

- **Bug report mơ hồ** — ngôn ngữ tự nhiên, thiếu chính xác, đôi khi sai.
- **Codebase lớn & phức tạp** — lỗi thường nằm **không** ở file được nhắc đến, mà lan truyền qua chuỗi gọi hàm (call chain).

Hệ quả: con người phải duyệt hàng trăm file → tốn thời gian, phụ thuộc kinh nghiệm, dễ bỏ sót.

Động lực: Quy trình truyền thống gặp bottleneck

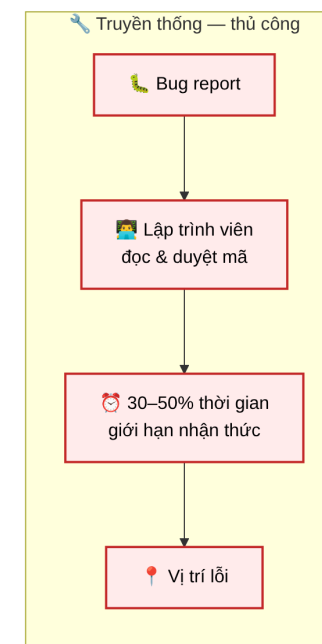
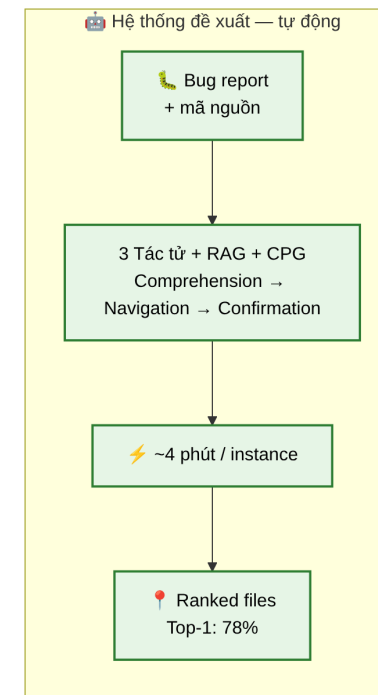
Truyền thống (thủ công):

- Lập trình viên đọc bug report
- Duyệt mã nguồn bằng từ khóa, grep, kinh nghiệm
- **Bottleneck:** 30–50% thời gian, giới hạn nhận thức con người

Hệ thống đề xuất (tự động):

- Đầu vào: bug report + đường dẫn repo
- 3 tác tử LLM + RAG + CPG
- ~4 phút / instance → **Top-1: 78%**

Mục tiêu: phục hồi có hệ thống vị trí lỗi, đặc biệt ở các case **ngoài tri thức ghi nhớ** của LLM.



Câu hỏi nghiên cứu & đóng góp

RQ chính: Một hệ đa tác tử LLM (với RAG + CPG) có tạo ra giá trị định vị *thực sự* so với bare-LLM không — hay chỉ đắt hơn mà không đúng hơn?

3 đóng góp của luận văn:

1. **E1 — Vòng lặp giả thuyết cạnh tranh:** sinh K giả thuyết lỗi, theo dõi niềm tin (belief tracking) bằng log-odds thuần Python.
2. **E2 — Khám phá ưu tiên (priority exploration):** tách bộ lập lịch (Python) khỏi LLM (chỉ chấm điểm), context $O(1)$ mỗi bước.
3. **E3 — Listwise rerank:** thu hẹp file→function + xếp lại top-K bằng 1 LLM call.

Cơ sở thực nghiệm: đánh giá đầy đủ trên **SWE-bench Lite, n=300**, so sánh paired với bare-LLM.

Phần II

Cơ sở lý thuyết & công trình liên quan

Các hướng tiếp cận định vị lỗi

Hướng	Đại diện	Đặc trưng	Hạn chế
IR-based	BugLocator, BLIA, BRTracer	Khớp từ vựng bug report ↔ mã nguồn (VSM, BM25)	Mù ngữ nghĩa; bỏ qua quan hệ cấu trúc
Learning-based	CNN, DeepRL, FastCode	Học nhúng kết hợp từ vựng + ngữ nghĩa	Cần dữ liệu huấn luyện; khó chuyển domain
Agentic (LLM)	LocAgent, BLAgent, Agentless, hệ thống đề xuất	LLM suy luận nhiều bước + công cụ + RAG	Chi phí cao; cần chứng minh giá trị

Hướng agentic đang nổi — tận dụng khả năng suy luận của LLM, nhưng đặt câu hỏi: *đáng giá không?* → luận văn trả lời bằng thực nghiệm.

Nền tảng kỹ thuật

- **LLM (Large Language Model):** mô hình ngôn ngữ lớn, suy luận qua prompt + tool-call. Đề tài dùng **qwen-plus** (Alibaba DashScope), giao tiếp chuẩn OpenAI → đổi provider được (OpenRouter, Gemini, Ollama/vLLM).
- **RAG (Retrieval-Augmented Generation):** bổ sung ngữ cảnh truy xuất (vector + BM25) vào prompt → giảm ảo giác, làm việc trên repo riêng.
- **Multi-Agent:** tách nhiệm vụ phức tạp thành nhiều tác tử chuyên biệt, mỗi tác tử có tool loop riêng → suy luận có cấu trúc.
- **Code Property Graph (CPG):** đồ thị AST + quan hệ gọi hàm/kế thừa/import → truy xuất cấu trúc, định vị lỗi lan truyền qua call chain.

Công trình liên quan (SOTA file-level)

Hệ thống	Backbone	Top-1	Đặc trưng
LocAgent	Claude-3.5	77.7	Graph-guided agent, nhiều công cụ
BLAgent	GPT-OSS-120B	78.6	Agentic RAG
Agentless	GPT-4o	63.0	Hai giai đoạn, không agent
CoSIL	Qwen2.5-32B	61.3	Iterative code graph search

Khoảng trống nghiên cứu:

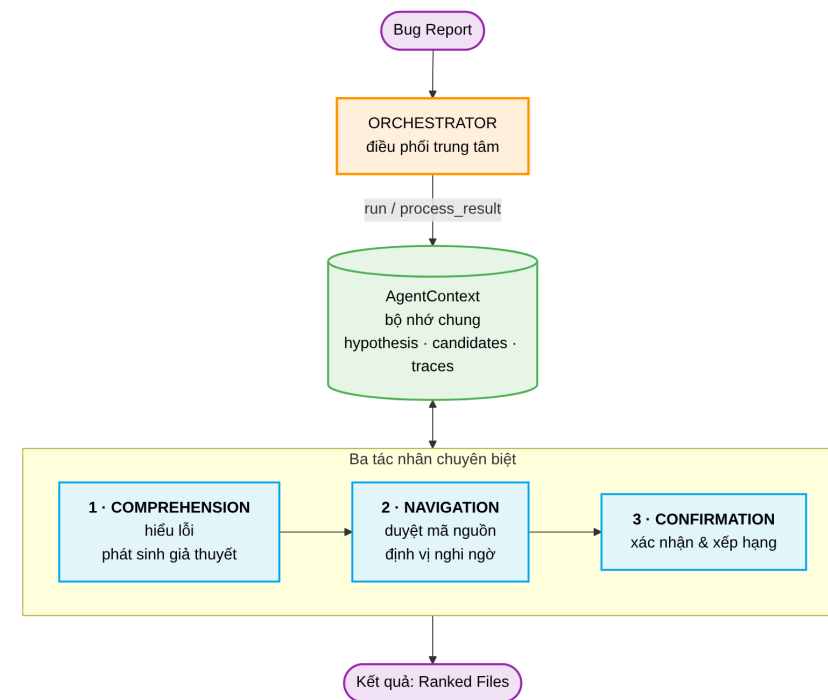
- Phần lớn báo cáo Top-1 mà **không so baseline bare-LLM cô lập** → khó quy trách giá trị cho agent.
- Ít hệ thống tách bạch **bộ lập lịch** khỏi LLM (chi phí tích lũy context).
- Thiếu **phân tích autopsy** case thất bại → khó biết cải thiện ở đâu.

Phần III

Kiến trúc hệ thống

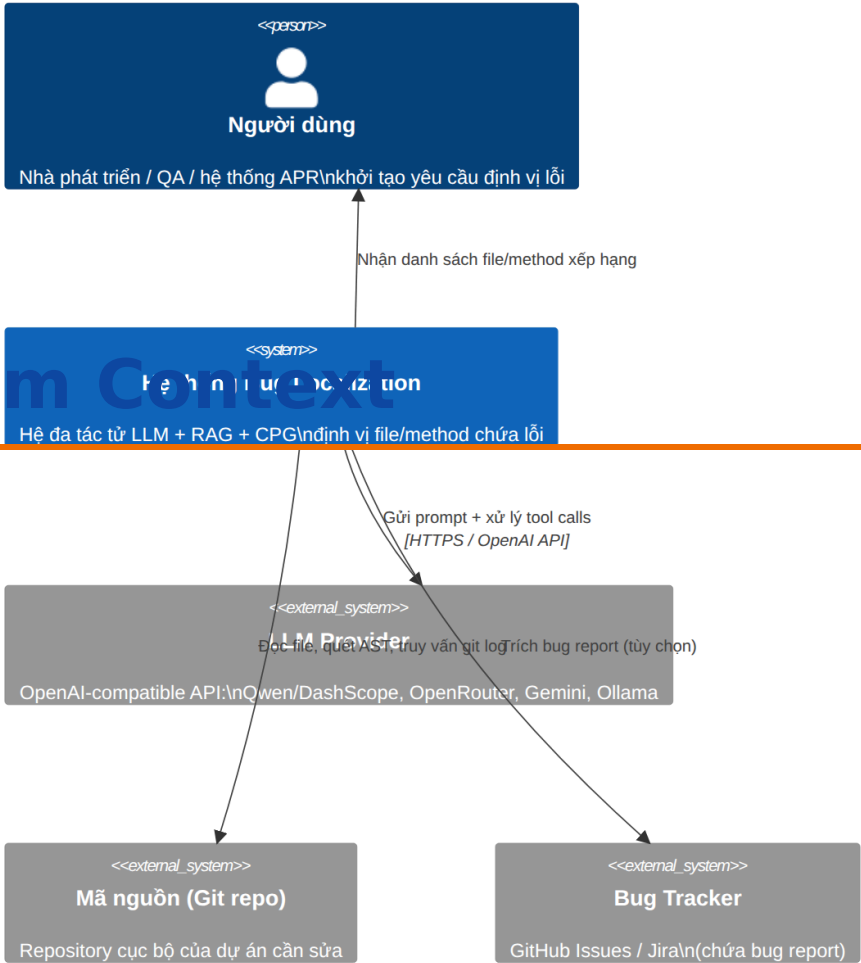
Tổng quan kiến trúc — 5 lớp chức năng

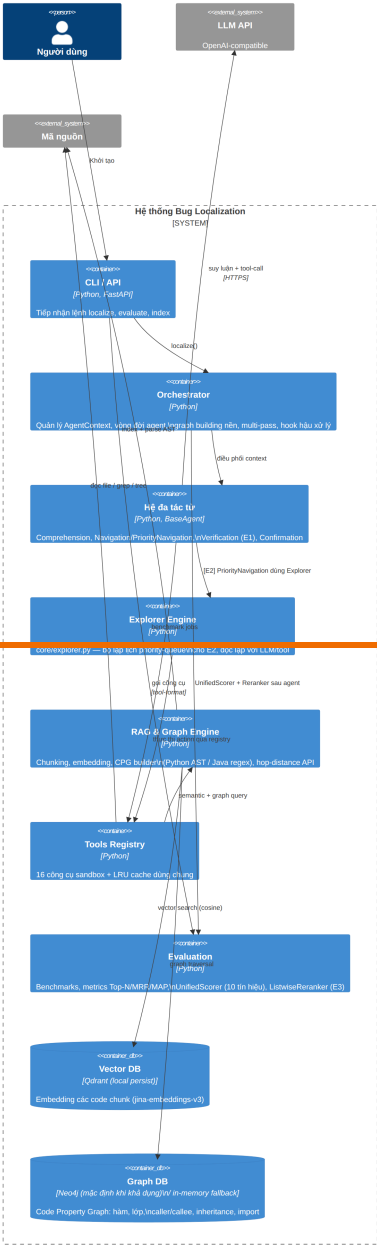
1. **Giao tiếp** (CLI / FastAPI) — `localize`, `evaluate`, `index`
 2. **Điều phối** (Orchestrator) — vòng đời tác tử, context chung, hook hậu xử lý
 3. **Tác tử** (Multi-Agent) — Comprehension → Navigation/Explorer → Confirmation + E1/E2
 4. **Tri thức** (RAG & CPG) — Qdrant vector store + Code Property Graph
 5. **Công cụ** (Tools) — 16 công cụ sandbox, chia sẻ cache LRU
- 3 đóng góp E1/E2/E3** đều có công tắc `.env` riêng, tắt mặc định, không đổi schema output → bật/tắt độc lập, tạo cơ sở cho phần **ablation** (Phần VI).



Hình 3.2 — System Context Diagram

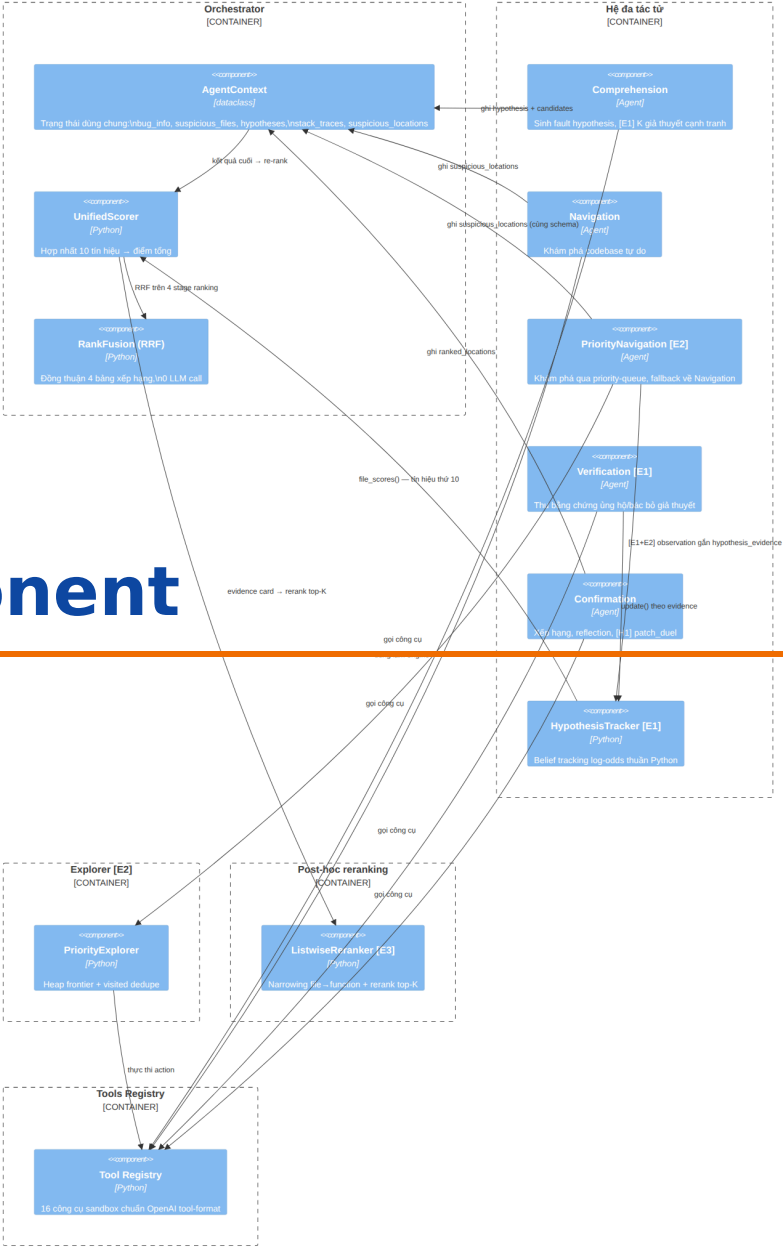
C4 Level 1 — System Context

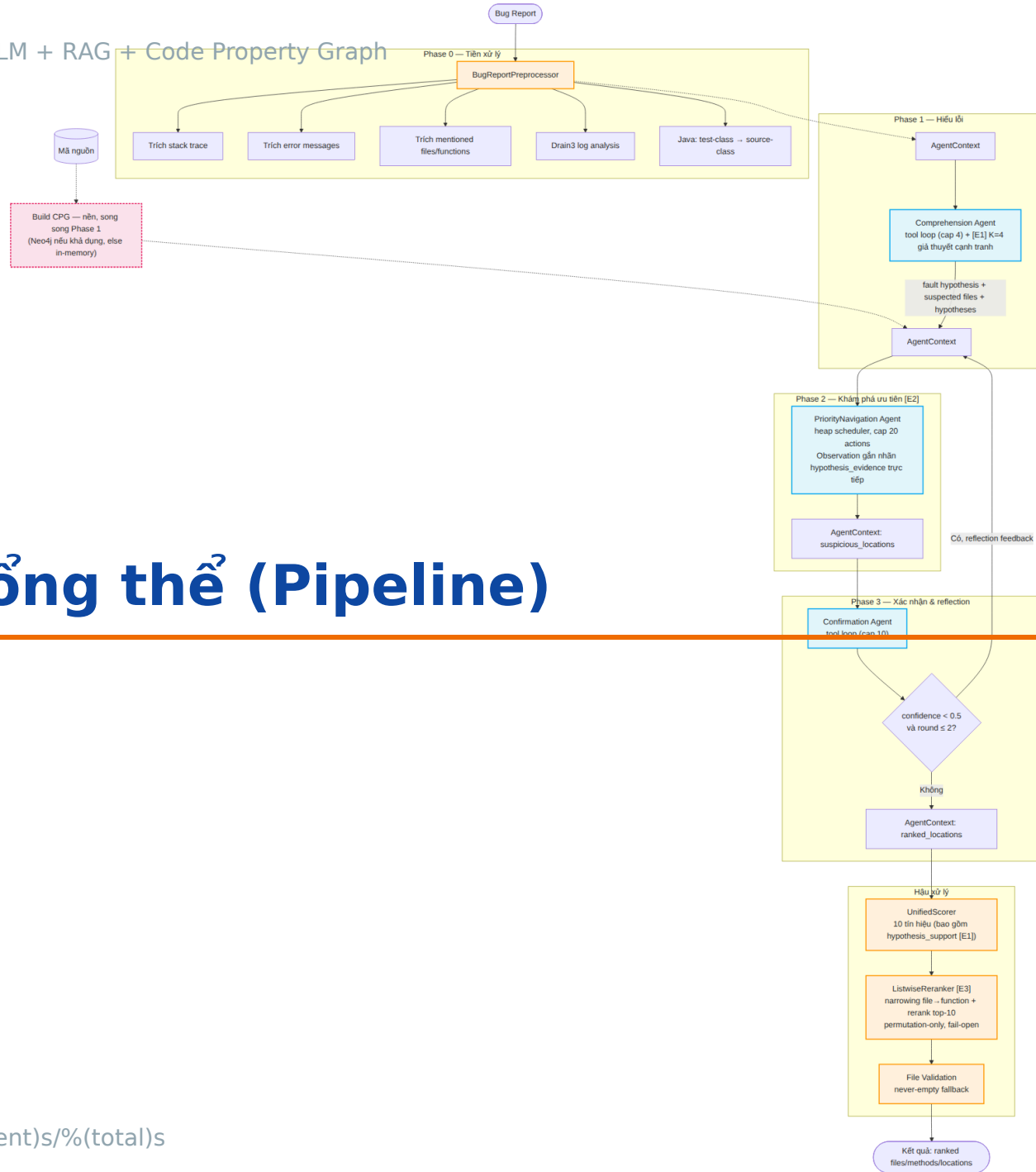




C4 Level 2 — Container

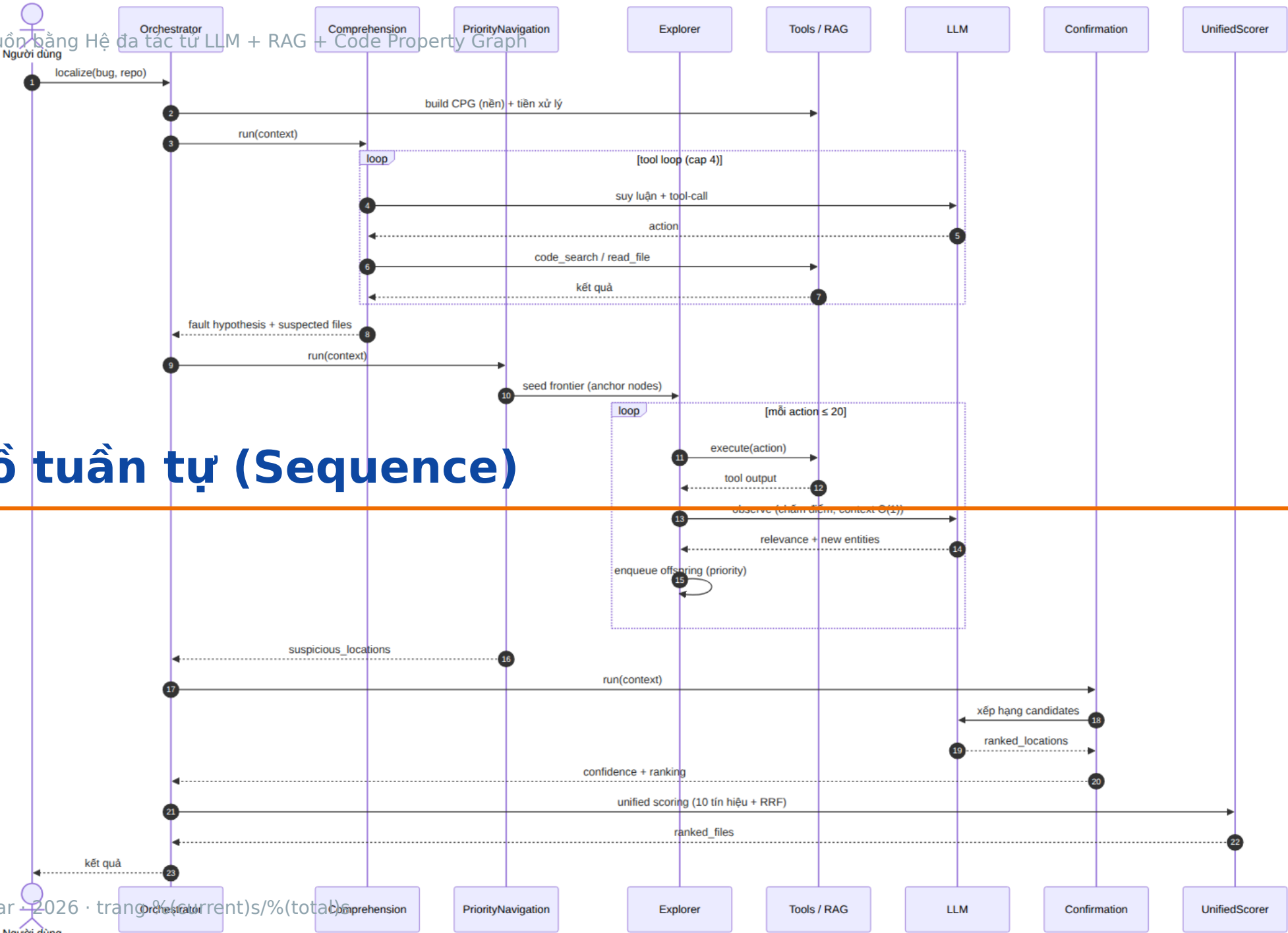
C4 Level 3 — Component





Luồng xử lý tổng thể (Pipeline)

Sơ đồ tuần tự (Sequence)



Ba tác tử chuyên biệt

Tác tử	Vai trò	Tools chính	Cap
1. Comprehension	Hiểu lỗi → sinh fault hypothesis + suspected files	code_search, read_file, list_directory, parse_logs	4
2. PriorityNavigation [E2]	Khám phá qua heap scheduler; LLM chỉ chấm điểm	+ semantic_search, graph_search, find_callers/callees	20
3. Confirmation	Xác nhận + xếp hạng candidates, reflection	code_search, read_file, find_callers/callees	10

Mọi tác tử kế thừa `BaseAgent` , chạy **agentic loop**: LLM → tool call → LLM → ... → JSON cuối. Đạt `max_iterations` mà chưa ra → ép trả lời (forced final answer).

E2 Priority Explorer — công thức ưu tiên

Ý tưởng OrcaLoca: một *frontier* (heap ưu tiên) các action; LLM chỉ chấm điểm relevance, scheduler tự quyết định thứ tự.

→ Tách "**quyết định đi đâu**" (Python, xác định) khỏi "**hiểu gì**" (LLM).

$$\text{priority}(t) = w_{\text{llm}} \cdot \text{rel}(t)/10 + w_{\text{graph}} \cdot \text{graph}(t) + w_{\text{signal}} \cdot \text{prior}(t)$$

Seed theo nguồn: stack trace (1.0) > mentioned (0.7) > hypothesis (0.6) > keyword (0.4)

Dừng: **EARLY_SUCCESS** (≥ 8 finding relevance ≥ 8) · **STAGNATION** (5 quan sát kém liên tiếp) ·

MAX_FINDINGS=15

Graph term: $\text{graph}(t) = 1 / (1 + \text{distance})$ — distance từ multi-source BFS (cache LRU 2048).

Unified Scorer — hợp nhất 10 tín hiệu

Sau khi agents chạy xong, `UnifiedScorer` cộng có trọng số **10 tín hiệu** cho mỗi file ứng viên:

Tín hiệu	Trọng số	Tín hiệu	Trọng số
Stack trace (decay 0.85)	2.5	Git recency (half-life 90d)	0.5
Error message match	1.5	Graph proximity	0.8
LLM confidence	1.0	Semantic similarity	0.6
Rank consensus (RRF)	1.0	Method count boost	0.3
Mentioned file	1.2	Hypothesis support [E1]	0.8

`test_file_penalty` (−0.5) trừ điểm file test trừ khi lỗi rõ ràng trong test setup.

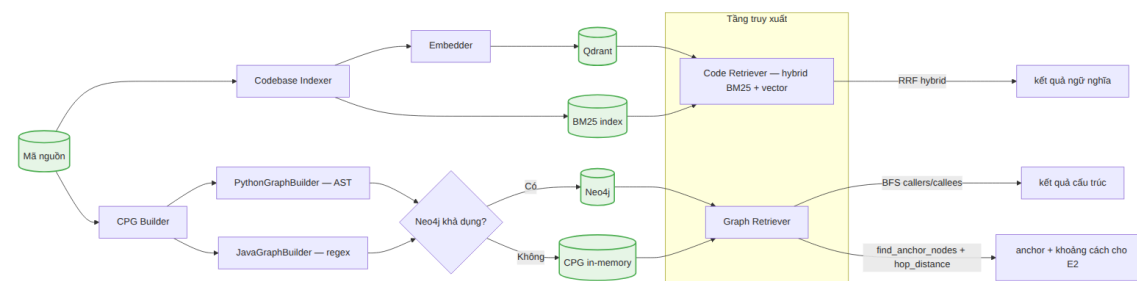
RAG & Code Property Graph

Vector store (Qdrant): `jina-embeddings-v3`, hybrid BM25 + semantic, hợp nhất bằng RRF (k=60).

Code Property Graph: 5 thực thể/quan hệ — hàm, lớp, chứa, import, gọi, kế thừa.

- `PythonGraphBuilder` — AST chính xác
- `JavaGraphBuilder` — regex (ít chính xác hơn → trọng số graph thấp hơn)

GraphRetriever: keyword anchor + BFS expansion → định vị file **không được nhắc** trong report (lỗi lan truyền qua call chain).



Phần IV

Đóng góp phương pháp — E1 / E2 / E3

Tổng quan 3 đóng góp

	Đóng góp	Vấn đề giải quyết	Đo lường
E1	Vòng lặp giả thuyết cạnh tranh	Agent dừng sớm ở hypothesis đầu tiên, mất khả năng phân định	Top-1 discrimination
E2	Khám phá ưu tiên	Tool loop tự do đốt context tích lũy, đắt & không hội tụ	Chi phí + recall
E3	Listwise rerank	Pointwise scoring sai thứ tự tương đối giữa candidates	Top-1 / Top-3

Triết lý chung: tách phần *xác định* (Python, reproducible) khỏi phần *hiểu* (LLM, đắt) → giảm phụ thuộc vào LLM cho quyết định algorithmic.

E1 Vòng lặp giả thuyết cạnh tranh + HypothesisTracker

Vấn đề: Agent thường cam kết sớm với fault hypothesis đầu tiên → khi có 2 file cùng subsystem, chọn sai (pattern "tầng liền kề").

Giải pháp:

- Sinh **K=4 giả thuyết lỗi cạnh tranh** trong Comprehension.
- `HypothesisTracker` theo dõi posterior mỗi hypothesis bằng **log-odds thuần Python** — cập nhật khi có evidence.
- LLM **chỉ gán nhãn** bằng chứng (ủng hộ / bác bỏ / trung tính); tính toán posterior là Python.

Tín hiệu đầu ra: `hypothesis_support` (tín hiệu thứ 10 trong Unified Scorer) + `reflection_summary()` cho vòng lặp reflection.

Lý do không để LLM tự cập nhật niềm tin: tin cậy kém, không reproducible, và tốn LLM call thừa.

E2 + **E3** Khám phá ưu tiên & Listwise rerank

E2 — PriorityNavigation (đã trình bày): heap frontier + LLM observer. Khi E2 bật, **VerificationAgent** (E1) bị bỏ qua vì Explorer tự gắn nhãn evidence trong **observe()**. Fallback về NavigationAgent nếu tìm quá ít.

E3 — ListwiseReranker (**evaluation/reranker.py**):

- Dựng **evidence card** cho top-K file (chỉ tín hiệu định tính — **không** mang điểm tổng hay thứ hạng → tránh anchoring).
- 1 LLM call xếp lại top-K, **permutation-only** (không đổi thành viên pool) → Top-10 / recall bất biến theo thiết kế.
- **Fail-open**: mọi lỗi parse → giữ thứ tự cũ, không sập.

Patch Duel (H1): 1 LLM call cuối phác thảo patch cho #1 vs #2 (giấu thứ hạng), chọn file sửa thật.
→ **đã đo n=300: KHÔNG có ý nghĩa thống kê, giữ tắt** (xem Phần VI).

Phần V

Thiết lập thí nghiệm

Benchmark & Metrics

Benchmark: SWE-bench Lite (`princeton-nlp/SWE-bench_Lite` , test split)

- **300 instance** đầu (để khớp subset cũ, so sánh apples-to-apples).
- Repo: django, sympy, sphinx, pytest, scikit-learn, pylint, flask, astropy...
- Ground truth = file sửa trong gold patch (`buggy_files`).

Metrics (file-level):

- **Top-1 / Top-3 / Top-5 accuracy** — file đúng có trong top-N?
- **MRR** (Mean Reciprocal Rank) — ưu tiên hạng cao.
- **MAP** (Mean Average Precision).

Dùng đúng các hàm metric của hệ thống (`top_n_accuracy` , `reciprocal_rank` , `average_precision` + chuẩn hóa `_paths_match`).

Baselines & kiểm định thống kê

Bare-LLM (baseline cô lập): 1 LLM call, không agent, không RAG, không tool — chỉ bug report + file tree → JSON xếp hạng.

Cấu hình	Đặc trưng
Bare-Legacy	prompt flat zero-shot (prompt gốc)
Bare-Research v1	CoT + delimiter + constrained decoding (prompt chuẩn nghiên cứu)
Bare-Research v2	v1 + ép ≥ 5 ứng viên

Backbone thống nhất: `qwen-plus` cho cả bare và agent (1 biến đổi duy nhất = khung agent).

Kiểm định: paired McNemar trên cùng 300 instance từng cặp → p-value cho delta Top-1/Top-5.

Temperature = 0 (deterministic) cho mọi hệ thống → loại nhiễu lấy mẫu.

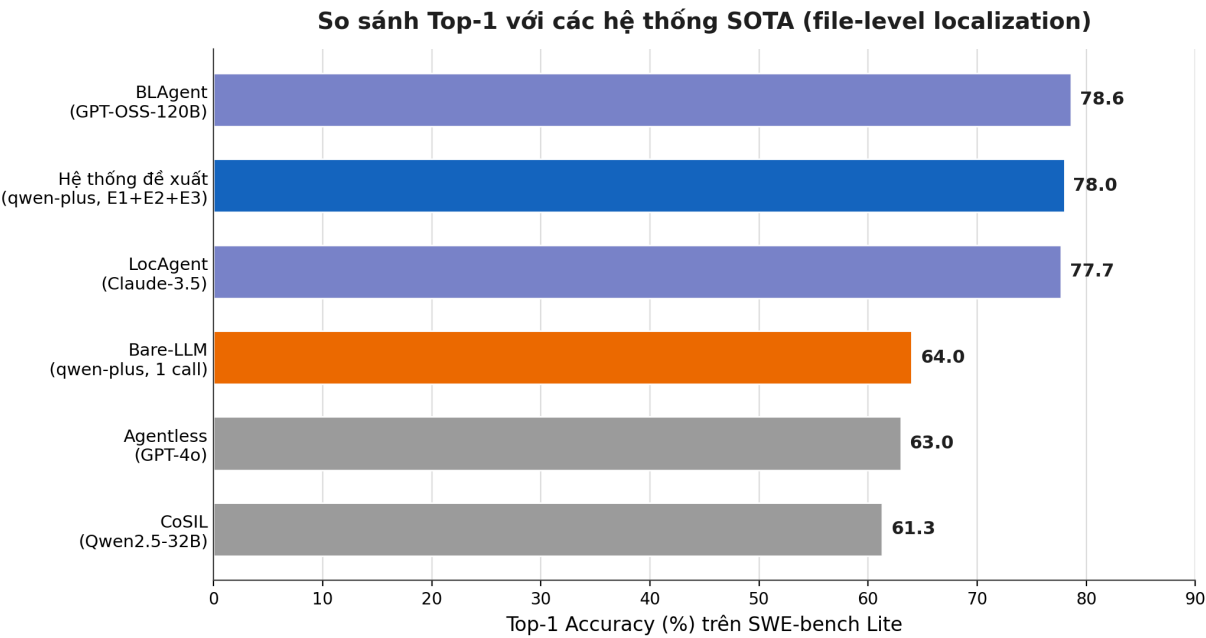
Phần VI

Kết quả & phân tích

Kết quả chính (n=300)

Hệ thống	Top-1	Top-3	Top-5	MRR
Bare-LLM (1 call)	64.0	83.7	87.0	0.738
Đề xuất (E1+E2+E3)	78.0	87.3	89.3	0.829

Top-1 ngang SOTA cùng phân khúc backbone (78.0 vs LocAgent 77.7, BLAgent 78.6). Top-3/5 thấp hơn LocAgent — do LocAgent dùng backbone Claude-3.5 mạnh hơn nhiều.

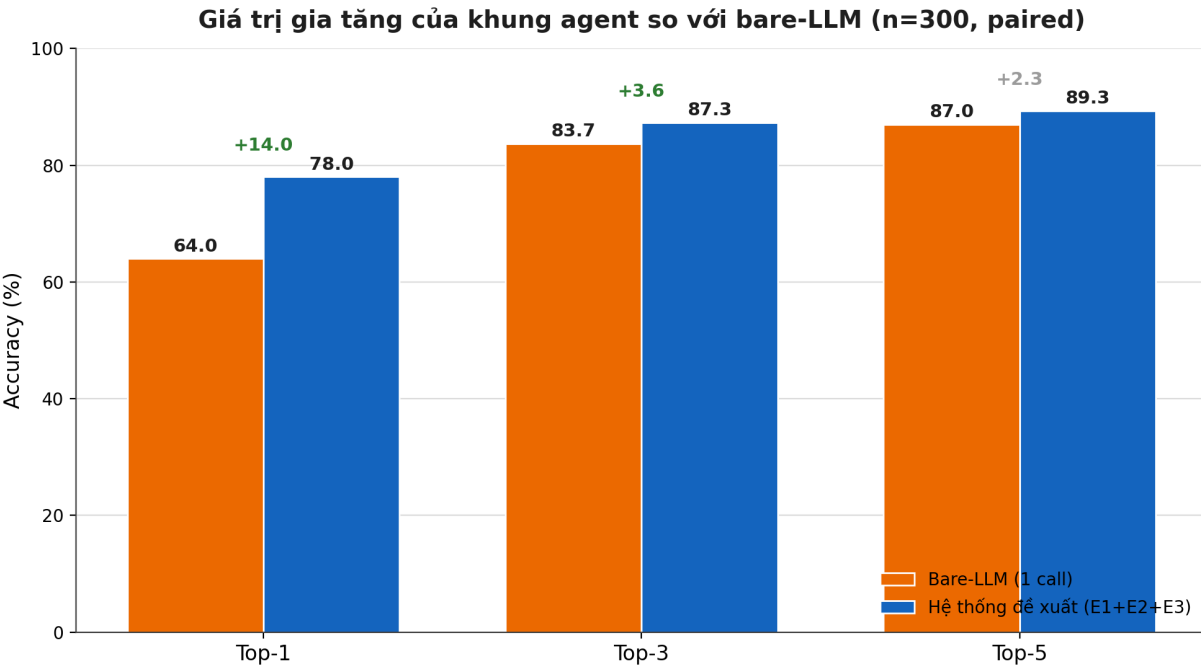


Claim chính: Giá trị gia tăng của khung agent

Metric	Bare	Đề xuất	Δ	McNemar p
Top-1	64.0	78.0	+14.0	1.4×10^{-6}
Top-3	83.7	87.3	+3.6	—
Top-5	87.0	89.3	+2.3	0.28 (không ý nghĩa)

Diễn giải: +14 điểm Top-1, **ý nghĩa rất mạnh** (59 case agent thắng, 17 bare thắng). Top-5 không có ý nghĩa — bare-LLM đã tiếp cận giới hạn recall nhờ **ghi nhớ** các repo phổ biến trong train.

Chi phí đổi lại: ~34× LLM calls, ~100× thời gian.



Bare-LLM baseline — CoT có giúp không?

Cấu hình (n=300)	Top-1	Top-3	Top-5	MRR	calls
Bare-Legacy (flat)	64.0	83.7	87.0	0.738	1
Bare-Research v1 (CoT)	67.7	79.0	79.7	0.731	1
Bare-Research v2 (ép ≥5)	60.7	74.0	76.3	0.674	1

2 phát hiện:

- 1. **CoT đẩy Top-1 (+3.7)** nhưng lại **giảm Top-3/Top-5** — CoT làm LLM *tự tin & kén chọn* (median 2 file/inst).
- 2. **Ép "≥5 ứng viên" (v2) là net-negative** — giảm MỌI metric kể cả Top-5. Padding ứng viên = noise, không lấp case miss.

→ Negative result quan trọng: "under-listing" KHÔNG phải vấn đề cốt lõi; giữ v1, bỏ v2.

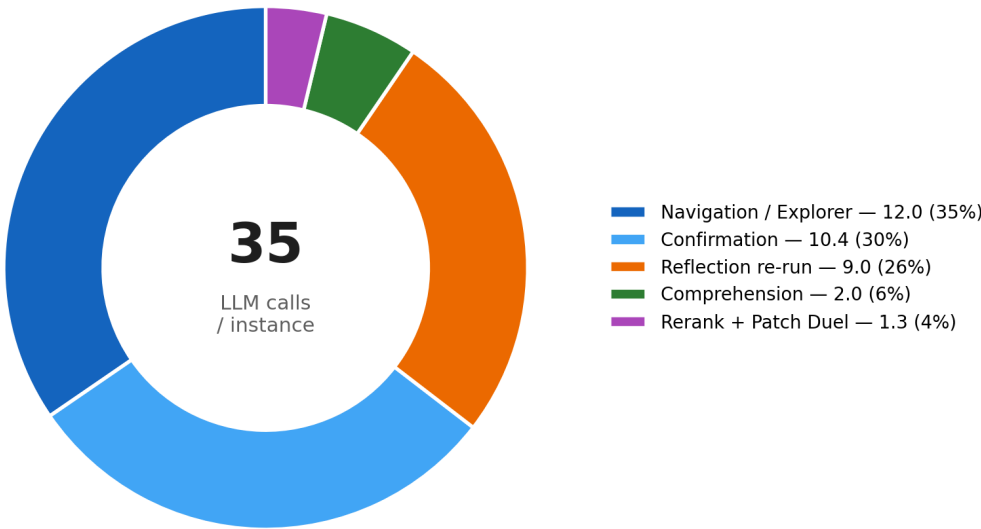
Phân tích chi phí LLM (≈34.7 calls/instance)

Thành phần	calls	%
Navigation / Explorer	~12	35%
Confirmation	~10.4	30%
Reflection re-run	~9	26%
Comprehension	~2	6%
Rerank + Duel	~1.3	4%

Phát hiện: 57% tổng calls là "cap-hits" (agent chạy hết vòng cho phép) — Confirmation 65%, Comprehension 87%. Đây là dấu hiệu vòng lặp đang **xác minh lặp**, không phải khám phá mới.

→ Định hướng tối ưu: nén cap verification trước.

Phân bố chi phí LLM (≈ 34.7 calls/instance)

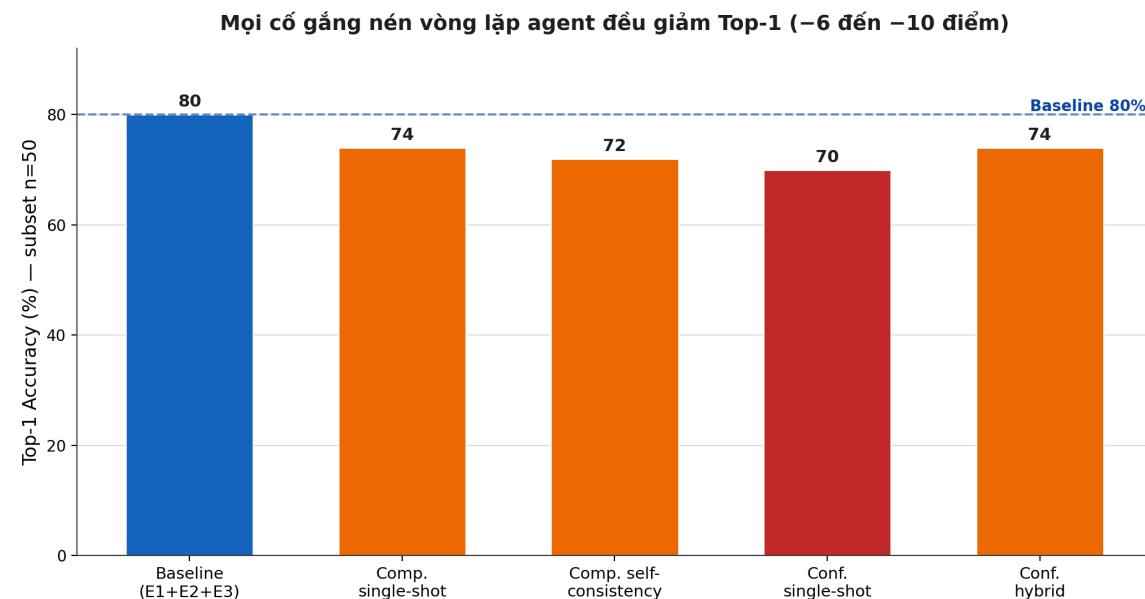


Ablation: Vòng lặp agent KHÔNG thể nén

6 thí nghiệm ablation (n=50, paired), mọi cố gắng nén vòng lặp agent cho cùng **chữ ký thất bại**:

- Top-3 / Top-5 **giữ hoặc tăng**
- Nhưng **Top-1 giảm 5-10 điểm**
- Mất cụm theo **repo django** (kiến trúc phân tầng: file triệu chứng $\neq$ file cần sửa)

Kết luận trần thuật chính: Vòng lặp xác minh lặp đi lặp lại của agent là cơ chế **chuyên biệt cho Top-1 discrimination**, không phải cho recall. Không thay được bằng single-pass listwise.



Autopsy: 22% case trượt Top-1

Cấu trúc thất bại (trên 300 baseline):

Bucket	Số case	%	Bản chất
Gold ở rank 2-5	34	11.3%	Vấn đề phân định (chọn sai file cùng subsystem)
Gold ngoài danh sách	29	9.7%	Vấn đề recall (gold không vào pool)
Gold ở rank 6-10	3	1.0%	Vấn đề xếp hạng

Trong đó 21 case có gold ở rank 2 (7%) — tiềm năng +7 điểm Top-1 nếu phân định đúng.

Pattern phổ biến: nhầm "tầng liền kề" cùng subsystem → options.py vs base.py , recorder.py vs executor.py .

Recall failure tập trung ở sympy (13) + django (10). 3 case predicted rỗng (sphinx) — bug FileValidation drop hết path.

Case study: Hai hệ thống thất bại BỔ SUNG

Instance	GT	Agent	Bare-Research	Bare-Legacy
astropy-14365	qdp.py	T1	T1	T1
flask-4992	config.py	T1	T1	T1
pylint-5859	misc.py	T1	T1	T1
django-10914	global_settings.py	✗	T1	T5
seaborn-2848	_oldcore.py	T1	✗	✗

django-10914 : LLM **đã thuộc** bug (bare T1 ngay), nhưng `listwise_rerank` của agent **làm hỏng** đáp án (rank 3→9). Pipeline tác dụng ngược trên bug đã memorize.

seaborn-2848 : LLM **không thuộc** (bare sai cả 2), agent nhờ exploration + RRF **tìm ra**. Pipeline thêm giá trị thật.

⇒ Agent mạnh ở bug cần khám phá code; bare mạnh (và rẻ 25×) ở bug đã nằm trong tri thức mô hình.

RRF + Patch Duel: Kết quả ÂM trên mẫu đủ lớn

Sau khi subset n=100 cho tín hiệu tích cực (+6đ), kiểm chứng trên **đầy đủ n=300**:

Metric	Baseline	RRF + Patch Duel	Δ	paired p
Top-1	78.0	76.3	-1.7	0.55 (nhiều)
Top-3	87.3	85.3	-2.0	0.39 (nhiều)
Top-5	89.3	89.3	0	1.00

Kết quả: NULL. Cải thiện trên subset 100 là **selection bias** + nhiễu LLM, không tổng quát hóa.

Attribution Patch Duel (n=300): swap đúng:sai = **1.8:1** (không phải 3:1 như subset từng báo).
Net +4 từ Duel, nhưng RRF đánh đổi mất ~5 → tổng ≈ 0.

→ Giữ H1 và RRF **dưới flag, tắt mặc định**. Minh bạch về negative result.

Phần VII

Thảo luận & kết luận

Threats to validity (Hạn chế)

Hạn chế	Tác động	Hướng xử lý
Memorization trên benchmark public	Bare Top-1 là <i>cận trên</i> (trùng ghi nhớ)	Nghiên cứu cross-model contamination (đang chờ API key)
Confound legacy (script cũ)	A/B chưa hoàn toàn 1-biến	Re-run legacy bằng script mới
300 instance đầu (không random)	Có thể thiên lệch theo repo	Mở rộng toàn split + per-repo breakdown
qwen-plus đơn model	Chưa rõ transfer prompt	Ablation cross-model (Qwen2.5-7B, Gemini)
Backbone chênh lệch vs SOTA	So sánh không hoàn toàn công bằng	Minh bạch trong báo cáo

Đặc biệt quan trọng: bare-LLM "biết" nhiều bug từ train → 64% Top-1 của bare **không đo thuần khả năng suy luận**. Điều này càng làm nổi bật giá trị agent (thêm đúng chỗ ngoài ghi nhớ).

Định hướng phát triển (Headroom)

Theo autopsy, 2 hướng cải thiện khả thi nhất chưa khai thác:

1. Tầng recall (29 case gold không vào pool, +10 điểm tiềm năng) ★ khả thi nhất

- BM25 + query augmentation (FastCode-style)
- Mở rộng pool ứng viên trước khi rerank

2. Phân định "tầng liên kề" (21 case rank-2, +7 điểm)

- Cơ chế hiểu "file nào patch sẽ sửa" mạnh hơn Patch Duel 1-call
- Multi-round repair simulation

3. Tối ưu chi phí — nén cap-hits (57% calls) mà không hại Top-1 (khó, theo ablation).

Cross-model contamination study đang chờ `OPENROUTER_API_KEY` để đo mức độ ghi nhớ chéo mô hình.

Đóng góp chính (Tóm tắt)

1. **Hệ đa tác tử + RAG + CPG** cho bug localization, với 3 đóng góp phương pháp E1/E2/E3 có thể bật/tắt độc lập → ablation nghiêm ngặt.
2. **Value-over-bare-LLM** được chứng minh thống kê: **+14 điểm Top-1, $p = 1.4 \times 10^{-6}$, $n=300$** — đóng góp cốt lõi, sạch thống kê.
3. **Top-1 ngang SOTA** cùng phân khúc backbone (78.0% vs LocAgent 77.7, BLAgent 78.6) dù backbone qwen-plus yếu hơn nhiều.
4. **Kết luận ablation**: vòng lặp agent là cơ chế chuyên biệt cho Top-1 discrimination — không thể nén bằng single-pass (6 thí nghiệm độc lập).
5. **Phân tích autopsy** + case study: failure mode của agent và bare-LLM **bổ sung cho nhau**; headroom cải thiện được lượng hóa.

Kết luận

Hệ đa tác tử LLM + RAG + CPG tạo ra **giá trị định vị thực sự** so với bare-LLM:

- **+14 điểm Top-1** ($p = 1.4 \times 10^{-6}$), **Top-1 ngang SOTA**
- Vòng lặp agent = cơ chế chuyên biệt cho **Top-1 discrimination**
- Thất bại của agent & bare-LLM **bổ sung cho nhau**
- Cơ chế E1/E2/E3 bật/tắt độc lập → **ablation có thể lặp lại**

Triết lý thiết kế: tách phần *xác định* (Python, reproducible) khỏi phần *hiểu* (LLM, đắt).

Xin cảm ơn các anh/chị/cô/thầy đã lắng nghe.

? Câu hỏi & Thảo luận

Dữ liệu thô & log đầy đủ:

- `results/swebench_300_e123_qwen.json` — baseline 300
- `results/cross_model/bare_qwen_300_final.json` — bare 300
- `results/ablation_*.json` — 7 thí nghiệm ablation

Tài liệu chi tiết:

- `docs/RESULTS_SUMMARY.md` — tổng hợp kết quả
- `docs/BARE_LLM_BASELINE_REPORT.md` — báo cáo baseline
- `docs/CHUONG_KIEN_TRUC.md` — Chương 3 kiến trúc

Xin cảm ơn!